International Islamic University Islamabad

Faculty of Engineering andTechnology

Department of Electrical and Computer Engineering

AI & Machine Learning Lab

Experiment No. 2: Implementing Numerical Data Operations Using NumPy

Name of Student: ……Bushra Nazir……

Registration No.: ….1071-F22……..

Date of Experiment: ….27-2-2026……

Submitted To: …..Engr.Asad……...

Objectives:

- Understand and manipulate NumPy arrays**,** including indexing, slicing, reshaping, and performing vectorized operations for efficient numerical computation.
- Process and transform numerical data using NumPy, including handling missing values, normalizing, standardizing, and preparing features for machine learning applications.
- Apply NumPy mathematical and statistical functions to analyze datasets and extract meaningful numerical patterns.

Equipment & Tools Required

Hardware:

- Personal Computer/Laptop with a minimum of 4GB RAM (8GB recommended)

Software & Tools:

- Jupyter Notebook (via Anaconda) or Google Colab
- Internet Access *(fordownloadingpackagesandusing Google Colab if needed)*

Introduction to NumPy

What is NumPy?

NumPy (Numerical Python) is a fundamental library for Python numerical computing. It provides powerful tools for handling large, multidimensional arrays and matrices and a collection of mathematical functions to operate on them efficiently. Developed as an open-source project, NumPy is widely used in scientific computing, data analysis, and machine learning (ML) due to its speed and ease of use compared to standard Python lists. It is often considered the backbone of numerical computing in Python and serves as the foundation for many other libraries such as Pandas, SciPy, Scikit-Learn, TensorFlow, and PyTorch**.**

Why NumPy for Machine Learning (ML)?

NumPy is essential for ML because it enables fast computation**,** efficient memory usage**,** and vectorized operations, making it significantly faster than Python lists. It provides

built-in mathematical functions for linear algebra, statistics, and random number generation, which are crucial for ML tasks. Additionally, NumPy integrates seamlessly with libraries like Pandas, TensorFlow, and Scikit-Learn, enhancing data preprocessing and model building.

Example Use Cases in ML:

✔**Linear Regression** –Solving matrix equations for optimal weights

✔**Deep Learning** –Performing tensor operations efficiently

✔**Data Preprocessing** –Handling missing values, feature scaling, and normalization

Installing NumPy

To use NumPy in Python, you first need to install it. NumPy can be installed using **pip**, which is the Python package manager. Run the following command in the terminal or command prompt:

```
PS C:\Users\ASAD EHSAN> pip install numpy
Requirement already satisfied: numpy in c:\users\asad ehsan\anaconda3\lib\site-packages (1.23.5)
```

After installation, the system will respond accordingly. If the library is already installed, it will show “Requirement already satisfied.”

If you are using Jupyter notebook, use

```
!pip install numpy
```

```
Requirement already satisfied: numpy in c:\users\asad ehsan\anaconda3\lib\site-packages (1.23.5)
```

Importing NumPy

Once NumPy is installed, you need to import it before using it in your Python scripts. The convention is to import NumPy using the alias np:

```
import numpy as np
```

This alias makes it easier to use NumPy functions throughout the code. For example, creating an array can be done as follows:

```
arr = np.array([1, 2, 3, 4, 5])
print(arr)
```

```
[1 2 3 4 5]
```

2. NumPy Basics

NumPy vs. Python Lists (Performance and Efficiency)

Python lists are flexible, but they are not optimized for numerical computing. Let's compare NumPy arrays with Python lists in terms of speed, memory usage, and operations.

1. Performance: NumPy is Faster than Lists

NumPy arrays are much faster than Python lists because they use C-optimized functions for computations. Let’s compare the speed of summing elements using Python lists and NumPy arrays:

```
import numpy as np
import time

# Using Python list
size = 1_000_000  # 1 million elements
py_list = list(range(size))

start_time = time.time()
sum_py = sum(py_list)
end_time = time.time()
print("Sum using Python list:", sum_py)
print("Time taken:", end_time - start_time, "seconds")

# Using NumPy array
np_array = np.arange(size)

start_time = time.time()
sum_np = np.sum(np_array)
end_time = time.time()
print("\nSum using NumPy array:", sum_np)
print("Time taken:", end_time - start_time, "seconds")
```

Output

```
Sum using Python list: 499999500000
Time taken: 0.035129547119140625 seconds

Sum using NumPy array: 1783293664
Time taken: 0.0020096302032470703 seconds
```

As seen above, NumPy performs significantly faster than Python lists when dealing with large datasets.

2. Memory Efficiency: NumPy Uses Less Memory

NumPy arraysconsume **less memory** comparedto Python lists because they store data in a contiguous memory block, while lists store pointers to objects.

```
import sys

# Python list memory usage
py_list = list(range(1000))
print("Python list size:", sys.getsizeof(py_list) * len(py_list), "bytes")

# NumPy array memory usage
np_array = np.arange(1000)
print("NumPy array size:", np_array.nbytes, "bytes")
```

```
Python list size: 8056000 bytes
NumPy array size: 4000 bytes
```

NumPy arrays take up much less memory than Python lists, making them ideal for handling large datasets.

3. Vectorized Operations: NumPy is More Efficient

Python lists require loops for mathematical operations, whereas NumPy can perform vectorized operations, eliminating the need for explicit loops. Example: Multiply each element in a list/array by 2.

```
py_list = [1, 2, 3, 4, 5]
py_result = [x * 2 for x in py_list]
print(py_result)
```

```
[2, 4, 6, 8, 10]
```

```
np_array = np.array([1, 2, 3, 4, 5])
np_result = np_array * 2
print(np_result)
```

```
[ 2  4  6  8 10]
```

The NumPy version is faster and requires less code.

Python lists are flexible but not optimized for numerical operations. NumPy provides ndarray (N-dimensional array), which is much faster, more memory-efficient, and supports vectorizedoperations .

Key Differences Between Lists and NumPy Arrays

Feature	Python List	NumPy Array (ndarray)
Performance	Slower (usesloops)	Faster (C-optimized)
Memory Usage	Higher (stores references)	Lower (contiguous memory)
Mathematical Operations	Requires explicit loops	Supports vectorized operations
Data Type Consistency	Can store mixed types	Homogeneous (same type)

Creating NumPy Arrays

NumPy provides various functions to create arrays efficiently.

1. Using np.array() – Convert a List to an Array

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])
print(arr)
print(type(arr))  # Output: <class 'numpy.ndarray'>
```

```
[1 2 3 4 5]
<class 'numpy.ndarray'>
```

2. Using np.zeros() – Create an Array of Zeros

Creates an array filled with zeros.

```
zeros_array = np.zeros((3, 3))  # 3x3 matrix of zeros
print(zeros_array)
```

```
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]
```

3. Using np.ones() – Create an Array of Ones

Creates an array filled with ones

```
ones_array = np.ones((2, 4))  # 2x4 matrix of ones
print(ones_array)
```

```
[[1. 1. 1. 1.]
 [1. 1. 1. 1.]]
```

4. Using np.arange() – Create a Range of Values

Works like Python's range() but returns an array.

```
arr = np.arange(1, 10, 2)  # Start at 1, go up to 10, step 2
print(arr)
```

```
[1 3 5 7 9]
```

5. Using np.linspace() – Create Evenly Spaced Values

Divides a range into equal parts.

```
arr = np.linspace(0, 5, 10)  # 10 values between 0 and 5
print(arr)
```

```
[0.         0.55555556 1.11111111 1.66666667 2.22222222 2.77777778
 3.33333333 3.88888889 4.44444444 5.        ]
```

Data Types in NumPy (dtype)

NumPy arrays have a single data type, making them faster than Python lists.

Checking the Data Type of an Array

```
arr = np.array([1, 2, 3])
print(arr.dtype)  # Output: int32 (or int64 depending on system)
```

```
int32
```

Specifying a Data Type

```
arr = np.array([1, 2, 3], dtype=np.float32)  # Create a float array
print(arr)
print(arr.dtype)  # Output: float32
```

```
[1. 2. 3.]
float32
```

Common NumPy Data Types

NumPy Data Type	Description
int32, int64	Integer numbers
float32, float64	Floating point numbers
bool_	Boolean values (True/False)
complex64	Complex numbers

Shape and Dimensions of Arrays

NumPy arrays have attributes to check their structure.

1. .shape – Get the Shape of an Array

Returns a tuple representing the number of rows and columns.

```
arr = np.array([[1, 2, 3], [4, 5, 6]])
print(arr.shape)  # Output: (2, 3) -> 2 rows, 3 columns
```

```
(2, 3)
```

.ndim – Get the Number of Dimensions

```
print(arr.ndim)  # Output: 2 (2D array)
```

```
2
```

.size – Get the Total Number of Elements

```
print(arr.size)  # Output: 6 (Total elements in array)
```

```
6
```

Reshaping and Flattening Arrays

1. .reshape() – Change the Shape of an Array

Modifies the array structure without changing its elements.

```
arr = np.arange(1, 7)  # [1 2 3 4 5 6]
reshaped_arr = arr.reshape((2, 3))  # Convert to 2 rows, 3 columns
print(reshaped_arr)
```

```
[[1 2 3]
 [4 5 6]]
```

2. .flatten() – Convert a Multi-Dimensional Array into 1D

```
flat_arr = reshaped_arr.flatten()
print(flat_arr)  # Output: [1 2 3 4 5 6]
```

```
[1 2 3 4 5 6]
```

3. NumPy Indexing & Slicing

Indexing and slicing are crucial for extracting and manipulating data in NumPy arrays. NumPy offers powerful and efficient ways to access elements, including basic indexing, slicing, Boolean masking, and fancy indexing.

Accessing Elements in NumPy Arrays

1. Accessing Elements in 1D Arrays

NumPyuses **zero-based indexing**,meaning the first element is at index 0 .

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50])
print(arr[0])  # Output: 10
print(arr[2])  # Output: 30
print(arr[-1]) # Output: 50 (last element)
```

```
10
30
50
```

2. Accessing Elements in 2D Arrays (Matrix Indexing)

For **2D arrays**, use arr[row, column] notation.

```
arr_2d = np.array([[1, 2, 3],
                   [4, 5, 6],
                   [7, 8, 9]])

print(arr_2d[0, 0])  # Output: 1  (First row, first column)
print(arr_2d[1, 2])  # Output: 6  (Second row, third column)
print(arr_2d[-1, -1])  # Output: 9 (Last row, last column)
```

```
1
6
9
```

3. Accessing Elements in 3D Arrays

For **3D arrays**, use arr[depth, row, column].

```
arr_3d = np.array([[[1, 2, 3], [4, 5, 6]],
                   [[7, 8, 9], [10, 11, 12]]])

print(arr_3d[0, 1, 2])  # Output: 6 (First depth, second row, third column)
print(arr_3d[1, 0, 1])  # Output: 8 (Second depth, first row, second column)
```

```
6
8
```

Slicing NumPy Arrays (: operator)

Slicing allows you to extract **subarrays** using the **:** operator.

1. Slicing 1D Arrays

Syntax: arr[start:stop:step]

- **Start**: Starting index (default = 0)
- **Stop**: Up to, but **not including**, this index
- **Step**: Step size (default = 1)

```
arr = np.array([10, 20, 30, 40, 50, 60, 70])

print(arr[1:5])   # Output: [20 30 40 50]
print(arr[:4])    # Output: [10 20 30 40] (Start from 0)
print(arr[2:])    # Output: [30 40 50 60 70] (Till end)
print(arr[::2])   # Output: [10 30 50 70] (Every 2nd element)
print(arr[::-1])  # Output: [70 60 50 40 30 20 10] (Reversed array)
```

```
[20 30 40 50]
[10 20 30 40]
[30 40 50 60 70]
[10 30 50 70]
[70 60 50 40 30 20 10]
```

2. Slicing 2D Arrays

```
arr_2d = np.array([[1, 2, 3, 4],
                   [5, 6, 7, 8],
                   [9, 10, 11, 12]])

print(arr_2d[0:2, 1:3])  # Output: [[2 3] [6 7]] (Rows 0-1, Columns 1-2)
print(arr_2d[:, 2])      # Output: [3 7 11] (All rows, 3rd column)
print(arr_2d[1, :])      # Output: [5 6 7 8] (2nd row, all columns)
```

```
[[2 3]
 [6 7]]
[ 3  7 11]
[5 6 7 8]
```

3. Slicing 3D Arrays

```
arr_3d = np.array([[[1, 2, 3], [4, 5, 6]],
                   [[7, 8, 9], [10, 11, 12]]])

print(arr_3d[:, :, 1])  # Output: [[2 5] [8 11]] (All depths, all rows, 2nd column)
```

```
[[ 2  5]
 [ 8 11]]
```

Boolean Masking and Filtering

Boolean masking helps **filter elements** based on conditions.

1. Using Boolean Conditions

```
arr = np.array([10, 20, 30, 40, 50])

mask = arr > 25
print(mask)        # Output: [False False  True  True  True]
print(arr[mask])   # Output: [30 40 50]
```

```
[False False  True  True  True]
[30 40 50]
```

2. Direct Condition in Indexing

```
print(arr[arr < 40])  # Output: [10 20 30] (All elements < 40)
```

```
[10 20 30]
```

3. Applying Conditions on 2D Arrays

```
arr_2d = np.array([[1, 2, 3],
                   [4, 5, 6],
                   [7, 8, 9]])

print(arr_2d[arr_2d % 2 == 0])  # Output: [2 4 6 8] (Only even numbers)
```

```
[2 4 6 8]
```

Fancy Indexing

Fancy indexing allows selecting multiple elements at once using lists or arrays.

1. Selecting Specific Elements

```
arr = np.array([10, 20, 30, 40, 50])

indices = [0, 2, 4]  # Select elements at positions 0, 2, and 4
print(arr[indices])  # Output: [10 30 50]
```

```
[10 30 50]
```

2. Fancy Indexing in 2D Arrays

```
arr_2d = np.array([[10, 20, 30],
                   [40, 50, 60],
                   [70, 80, 90]])

rows = [0, 2]  # Select rows 0 and 2
cols = [1, 2]  # Select columns 1 and 2

print(arr_2d[rows, cols])  # Output: [20 90] (Elements at (0,1) and (2,2))
```

```
[20 90]
```

3. Selecting Entire Rows or Columns

```
print(arr_2d[[0, 2], :])  # Select rows 0 and 2, all columns
print(arr_2d[:, [0, 2]])  # Select all rows, columns 0 and 2
```

```
[[10 20 30]
 [70 80 90]]
[[10 30]
 [40 60]
 [70 90]]
```

4. NumPy Operations

NumPy provides a wide range of operations that make mathematical computations efficient and fast. These include element-wise operations, broadcasting, aggregate functions, and sorting/searching operations.

1. Element-wise Operations

Element-wise operations allow performing mathematical computations directly on arrays. These operations are applied to each corresponding element of the array.

Basic Arithmetic Operations

NumPy supports standard arithmetic operations such as addition (+), subtraction (-), multiplication (*), division (/), and exponentiation (**).

```python
import numpy as np

arr1 = np.array([1, 2, 3, 4])
arr2 = np.array([5, 6, 7, 8])

print(arr1 + arr2)  # Output: [ 6  8 10 12]
print(arr1 - arr2)  # Output: [-4 -4 -4 -4]
print(arr1 * arr2)  # Output: [ 5 12 21 32]
print(arr1 / arr2)  # Output: [0.2 0.33333333 0.42857143 0.5]
print(arr1 ** 2)    # Output: [ 1  4  9 16] (Exponentiation)
```

```
[ 6  8 10 12]
[-4 -4 -4 -4]
[ 5 12 21 32]
[0.2        0.33333333 0.42857143 0.5       ]
[ 1  4  9 16]
```

Operations Between Arrays and Scalars

Operations with scalars are broadcasted across all elements.

```python
arr = np.array([10, 20, 30, 40])

print(arr + 5)  # Output: [15 25 35 45]
print(arr * 2)  # Output: [20 40 60 80]
```

```
[15 25 35 45]
[20 40 60 80]
```

2. Broadcasting in NumPy

Broadcasting allows NumPy to perform operations on arrays of different shapes by expanding them to a compatible shape without copying data.

Example 1: Broadcasting with Scalars

```python
arr = np.array([[1, 2, 3],
                [4, 5, 6]])

print(arr + 10)
```

```
[[11 12 13]
 [14 15 16]]
```

Example 2: Broadcasting in Different Shaped Arrays

```
arr1 = np.array([[1, 2, 3],
                 [4, 5, 6]])
arr2 = np.array([10, 20, 30])

print(arr1 + arr2)
```

```
[[11 22 33]
 [14 25 36]]
```

Example 3: Incompatible Shapes (Error)

```
arr1 = np.array([[1, 2, 3],
                 [4, 5, 6]])
arr2 = np.array([10, 20])  # Shape mismatch

print(arr1 + arr2)  # This will raise a ValueError
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[61], line 5
      1 arr1 = np.array([[1, 2, 3],
      2                  [4, 5, 6]])
      3 arr2 = np.array([10, 20])  # Shape mismatch
----> 5 print(arr1 + arr2)

ValueError: operands could not be broadcast together with shapes (2,3) (2,)
```

For broadcasting towork, the shapes must be compatible.

Broadcasting Rule:

- Dimensionsmust be equal, or
- One of themmust be 1 for it to be broadcasted

3. Aggregate Functions

NumPy provides a variety of built-in aggregation functions that allow computing statistics over arrays.

1. Summation: np.sum()

Computes the sum of all elements.

```
arr = np.array([[1, 2, 3],
                [4, 5, 6]])

print(np.sum(arr))        # Output: 21 (Sum of all elements)
print(np.sum(arr, axis=0)) # Output: [5 7 9] (Column-wise sum)
print(np.sum(arr, axis=1)) # Output: [ 6 15] (Row-wise sum)
```

```
21
[5 7 9]
[ 6 15]
```

2. Mean: np.mean()

Computes the average value.

```
print(np.mean(arr))  # Output: 3.5 (Mean of all elements)
```

```
3.5
```

3. Standard Deviation & Variance

Measures how much values deviate from the mean.

```
print(np.std(arr))  # Output: 1.7078 (Standard deviation)
print(np.var(arr))  # Output: 2.9167 (Variance)
```

```
1.707825127659933
2.9166666666666665
```

4. Minimum & Maximum Values

Finds the smallest and largest element.

```
print(np.min(arr))  # Output: 1
print(np.max(arr))  # Output: 6
```

```
1
6
```

5. Finding Index of Min/Max Values

```
print(np.argmin(arr))  # Output: 0 (Index of minimum element)
print(np.argmax(arr))  # Output: 5 (Index of maximum element)
```

```
0
5
```

4. Sorting and Searching in NumPy

Sorting and searching help in organizing and retrieving elements efficiently.

1. Sorting Arrays (np.sort())

Sorts an array in ascending order.

```
arr = np.array([30, 10, 20, 50, 40])

print(np.sort(arr))  # Output: [10 20 30 40 50]
```

```
[10 20 30 40 50]
```

For 2D arrays:

```
arr_2d = np.array([[30, 10, 50],
                   [20, 40, 60]])

print(np.sort(arr_2d, axis=0))
print(np.sort(arr_2d, axis=1))
```

```
[[20 10 50]
 [30 40 60]]
[[10 30 50]
 [20 40 60]]
```

2. Indices of Sorted Elements (np.argsort())

Returns the indices that would sort an array.

```
arr = np.array([30, 10, 20, 50, 40])

print(np.argsort(arr))  # Output: [1 2 0 4 3]
```

```
[1 2 0 4 3]
```

3. Searching with np.where()

Finds **positions of elements** that satisfy a condition.

```
arr = np.array([10, 20, 30, 40, 50])

print(np.where(arr > 25))  # Output: (array([2, 3, 4]),)
```

```
(array([2, 3, 4], dtype=int64),)
```

5. Linear Algebra with NumPy

NumPy provides a powerful linear algebra module (numpy.linalg) that allows performing operations on vectors, matrices, and tensors efficiently. Linear algebra is the foundation of many machine learning algorithms, including regression, dimensionality reduction, and neural networks.

1. Vectors and Matrices in NumPy

Vectors (1D Arrays)

A vector is a one-dimensional array in NumPy.

```
import numpy as np
vector = np.array([1, 2, 3])
print(vector)
```

```
[1 2 3]
```

Matrices (2D Arrays)

A matrix is a two-dimensional array in NumPy.

```
matrix = np.array([[1, 2, 3],
                   [4, 5, 6]])
print(matrix)
```

```
[[1 2 3]
 [4 5 6]]
```

You can also create matrices using functions like np.zeros(), np.ones(), and np.eye() (identity matrix).

```
zero_matrix = np.zeros((3, 3))  # 3x3 zero matrix
identity_matrix = np.eye(3)  # 3x3 identity matrix

print(zero_matrix)
print(identity_matrix)
```

```
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

2. Matrix Multiplication in NumPy

NumPy provides multiple ways to perform matrix multiplication:

1. Using np.dot()

Computes the dot product of two arrays.

```
A = np.array([[1, 2],
              [3, 4]])
B = np.array([[5, 6],
              [7, 8]])

result = np.dot(A, B)
print(result)
```

```
[[19 22]
 [43 50]]
```

2. Using @ Operator (Preferred Method)

The @ operator performs matrix multiplication in a more readable way

```
result = A @ B
print(result)
```

```
[[19 22]
 [43 50]]
```

3. Element-wise Multiplication (* Operator)

If you use *, it performs element-wise multiplication instead of matrix multiplication.

```
print(A * B)
```

```
[[ 5 12]
 [21 32]]
```

3. Transpose of a Matrix (.T)

The transpose of a matrix swaps its rows and columns.

```
A = np.array([[1, 2, 3],
              [4, 5, 6]])

print(A.T)
```

```
[[1 4]
 [2 5]
 [3 6]]
```

4. Inverse of a Matrix (np.linalg.inv())

The inverse of a matrix is a key operation in solving linear equations. It is defined only for square, non-singular matrices (determinant ≠ 0).

```
A = np.array([[4, 7],
              [2, 6]])

A_inv = np.linalg.inv(A)
print(A_inv)
```

```
[[ 0.6 -0.7]
 [-0.2  0.4]]
```

To verify the inverse, multiplyingA with A_inv should give the identity matrix

```
print(A @ A_inv)
```

```
[[ 1.00000000e+00  0.00000000e+00]
 [-2.22044605e-16  1.00000000e+00]]
```

```
print(np.round(A @ A_inv, decimals=0))
```

```
[[ 1.  0.]
 [-0.  1.]]
```

5. Eigenvalues and Eigenvectors (np.linalg.eig())

Eigenvalues and eigenvectors are fundamental in dimensionality reduction (e.g., PCA in Machine Learning).

For a given square matrix A, the eigenvalues λ and eigenvectors v satisfy the equation:

$$Av=\lambda v$$

Finding Eigenvalues and Eigenvectors in NumPy

```
A = np.array([[4, -2],
              [1, 1]])

eigenvalues, eigenvectors = np.linalg.eig(A)

print("Eigenvalues:", eigenvalues)
print("Eigenvectors:\n", eigenvectors)
```

```
Eigenvalues: [3. 2.]
Eigenvectors:
 [[0.89442719 0.70710678]
 [0.4472136  0.70710678]]
```

Each **eigenvector** corresponds to an **eigenvalue**,andtheyhave many applications in **machine learning, image processing, and stability analysis**

6. Data Processing with NumPy

Data processing is a crucial step in Machine Learning (ML) and Data Science. Before feeding data into ML models, we need to **clean, scale, and transform** it into a suitable

format. NumPy provides powerful tools to handle missing values, normalize data, scale features, and apply encoding techniques.

1. Handling Missing Values in NumPy

In real-world datasets, missing values **(**NaN – Not a Number**)** are common. NumPy provides built-in functions to detect, handle, and replace NaNs.

1.1 Identifying Missing Values (np.nan, np.isnan())

```
import numpy as np

data = np.array([10, 20, np.nan, 40, np.nan, 60])
print("Data:", data)

# Identify NaN values
nan_mask = np.isnan(data)
print("NaN Mask:", nan_mask)
```

```
Data: [10. 20. nan 40. nan 60.]
NaN Mask: [False False  True False  True False]
```

1.2 Replacing Missing Values (np.nanmean())

A common strategy is to replace NaNs with the mean of the existing values.

```
mean_value = np.nanmean(data)  # Mean excluding NaNs
data[np.isnan(data)] = mean_value  # Replace NaNs with mean
print("Processed Data:", data)
```

```
Processed Data: [10.  20.  32.5 40.  32.5 60. ]
```

2. Normalization and Standardization

Machine learning models work best when numerical features are on a similar scale**.** Normalization and standardization are common techniques for scaling.

2.1 Normalization (Scaling between 0 and 1)

Normalization transforms the data into a range [0, 1] using:

$$X_{norm} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

```
data = np.array([10, 20, 30, 40, 50])

normalized_data = (data - np.min(data)) / (np.max(data) - np.min(data))
print("Normalized Data:", normalized_data)
```

```
Normalized Data: [0.   0.25 0.5  0.75 1.  ]
```

Why Normalize? It ensures all features have equal weight in ML models, used in neuralnetworks (e.g **.,** image pixelscaling **).**

2.2 Standardization (Z-score Normalization)

Standardization scales the data to have **a** meanof 0 **and** standard deviation of 1:

$$X_{std} = \frac{X - \mu}{\sigma}$$

```
mean = np.mean(data)
std_dev = np.std(data)

standardized_data = (data - mean) / std_dev
print("Standardized Data:", standardized_data)
```

```
Standardized Data: [-1.41421356 -0.70710678  0.          0.70710678  1.41421356]
```

When to Use Standardization? When data follows a normal distribution **(**e.g.**,** logistic regression, PCA**),** Helps ML models converge faster in gradient descent-based algorithms**.**

3. Feature Scaling using NumPy

Feature scaling ensures all input variables contribute **equally** to the model. The two most common methods are:

3.1 MinMax Scaling (Same as Normalization)

```
def min_max_scaler(data):
    return (data - np.min(data)) / (np.max(data) - np.min(data))

scaled_data = min_max_scaler(data)
print("MinMax Scaled Data:", scaled_data)
```

```
MinMax Scaled Data: [0.   0.25 0.5  0.75 1.  ]
```

3.2 StandardScaler (Z-score Scaling)

```
def standard_scaler(data):
    return (data - np.mean(data)) / np.std(data)

scaled_data = standard_scaler(data)
print("Standard Scaled Data:", scaled_data)
```

```
Standard Scaled Data: [-1.41421356 -0.70710678  0.          0.70710678  1.41421356]
```

MinMax Scaling is useful when data has known upper & lower bounds. Standard Scaling is useful for algorithms like PCA, SVM, K-Means, and linear regression.

4. One-Hot Encoding using NumPy

Many ML algorithms require categorical variables to be converted into numerical format. One-Hot Encoding (OHE) is a common method.

Example

```
categories = np.array(["Apple", "Banana", "Apple", "Orange", "Banana"])
unique_categories = np.unique(categories)
print("Unique Categories:", unique_categories)
```

```
Unique Categories: ['Apple' 'Banana' 'Orange']
```

EXERCISE

1. Convert the following list into a NumPy array and find its mean, median, and standard deviation: data = [15, 22, 13, 45, 37, 18, 21, 30]
2. Create a 2D NumPy array (3×3) filled with random integers between 1 and 100.
3. Consider an ML dataset where feature values range from 1 to 255 (such as image pixel values). Normalize the dataset using NumPy so that all values fall between 0 and 1.
4. Implement One-Hot Encoding manually using NumPy for the following categorical labels: labels = ['cat', 'dog', 'fish', 'dog', 'fish', 'cat']
5. Create the following arrays using NumPy functions:
    - A 1D array of numbers from 1 to 10.
    - A 2D array (3×3) filled with ones.
    - A 3D array with random numbers.
6. Generate a 5×5 matrix where the diagonal elements are 1 and all other elements are 0.
7. Reshape into a 2D matrix: arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9]) into shape (3,3).
8. A dataset contains 100,000 samples and 50 features stored as a NumPy array. You need to reshape it into mini batches of 100 samples. Implement this in NumPy.
9. Given the following NumPy array: arr = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90, 100]) Extract elements from index 2 to 6.
10. Create a 5×5 NumPy matrix with values from 1 to 25. Extract:
    - The second row.
    - The third column.
    - The bottom-right 2×2 submatrix.
11. Multiply each element of an array by 5 without using loops.
12. Given: arr1 = np.array([1, 2, 3]) arr2 = np.array([4, 5, 6]) Compute element-wise sum, difference, multiplication, and division.
13. Create a 10×10 NumPy array of random values and subtract the row-wise mean using broadcasting.
14. Implement element-wise multiplication of two matrices using broadcasting.
15. Implement Mean Squared Error (MSE) using NumPy:

y_true = np.array([3.0, -0.5, 2.0, 7.0]) y_pred = np.array([2.5, 0.0, 2.0, 8.0])

16. Identify and replace missing values (NaN) in the following NumPy array with the mean: data = np.array([1, 2, np.nan, 4, 5, np.nan, 7])
17. Implement Min-Max Scaling on the following array: arr = np.array([10, 20, 30, 40, 50]) Normalize values between 0 and 1.

---

1. Convert list into NumPy array and find mean, median, std

```python
# task_01
import numpy as np

data = [15, 22, 13, 45, 37, 18, 21, 30]
arr = np.array(data)

mean = np.mean(arr)
median = np.median(arr)
std = np.std(arr)

print("Mean =", mean)
print("Median =", median)
print("Standard Deviation =", std)
```

```
Mean = 25.125
Median = 21.5
Standard Deviation = 10.52897787061973
```

2. Create a 2D NumPy array (3×3) filled with random integers between 1 and 100.

```python
# task_02

import numpy as np

array = np.random.randint(1, 100, (3,3))

print(array)
```

```
[[52  2 39]
 [76 53 62]
 [24 59 88]]
```

3. Consider an ML dataset where feature values range from 1 to 255 (such as image pixel values). Normalize the dataset using NumPy so that all values fall between 0 and 1

```python
# task_03

data = np.array([1, 50, 100, 200, 255])

normalized = (data - np.min(data)) / (np.max(data) - np.min(data))

print(normalized)
```

```
[0.         0.19291339 0.38976378 0.78346457 1.        ]
```

4. Implement One-Hot Encoding manually using NumPy for the following categorical labels: labels = ['cat', 'dog', 'fish', 'dog', 'fish', 'cat']

```python
# task_04

import numpy as np

labels = np.array(['cat','dog','fish','dog','fish','cat'])

unique_labels = np.unique(labels)

print("Unique Labels:", unique_labels)
onehot = np.eye(len(unique_labels))

print(onehot)
```

```
Unique Labels: ['cat' 'dog' 'fish']
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

5. Create the following arrays using NumPy functions:

- A 1D array of numbers from 1 to 10.
- A 2D array (3×3) filled with ones.
- A 3D array with random numbers.

```python
# task_05
# A
import numpy as np
a = np.arange(1,11)
print(a)

# B
b = np.ones((3,3))
print(b)

# C
c = np.random.rand(2,2,2)
print(c)
```

```
[ 1  2  3  4  5  6  7  8  9 10]
[[1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]]
[[[0.38559673 0.2124768 ]
  [0.78622781 0.13689422]]

 [[0.78683158 0.96635881]
  [0.56186565 0.00536743]]]
```

6. Generate a 5×5 matrix where the diagonal elements are 1 and all other elements are 0.

```
# task_06

matrix = np.eye(5)
print(matrix)
```

```
[[1. 0. 0. 0. 0.]
 [0. 1. 0. 0. 0.]
 [0. 0. 1. 0. 0.]
 [0. 0. 0. 1. 0.]
 [0. 0. 0. 0. 1.]]
```

7. Reshape into a 2D matrix: arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9]) into shape (3,3).

```
# task_07

arr = np.array([1,2,3,4,5,6,7,8,9])

reshaped = arr.reshape(3,3)

print(reshaped)
```

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

8. A dataset contains 100,000 samples and 50 features stored as a NumPy array. You need to reshape it into mini batches of 100 samples. Implement this in NumPy

```
data = np.random.rand(100000,50)

batches = data.reshape(1000,100,50)

print(batches.shape)
```

```
(1000, 100, 50)
```

9. Given the following NumPy array: arr = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90, 100]) Extract elements from index 2 to 6.

```
# task_09

arr = np.array([10,20,30,40,50,60,70,80,90,100])

result = arr[2:7]

print(result)
```

```
[30 40 50 60 70]
```

10.Create a 5×5 NumPy matrix with values from 1 to 25. Extract:

- The second row.
- The third column.
- The bottom-right 2×2 submatrix.

```
# task_10

matrix = np.arange(1,26).reshape(5,5)

second_row = matrix[1]
third_column = matrix[:,2]
submatrix = matrix[3:5,3:5]

print("Second Row:",second_row)
print("Third Column:",third_column)
print("Submatrix:\n",submatrix)
```

```
Second Row: [ 6  7  8  9 10]
Third Column: [ 3  8 13 18 23]
Submatrix:
 [[19 20]
 [24 25]]
```

11.Multiply each element of an array by 5 without using loops.

```
# task_11

arr = np.array([1,2,3,4,5])

result = arr*5

print(result)
```

```
[ 5 10 15 20 25]
```

12.Given: arr1 = np.array([1, 2, 3]) arr2 = np.array([4, 5, 6]) Compute element-wise sum, difference, multiplication, and division

```
arr1 = np.array([1,2,3])
arr2 = np.array([4,5,6])

sum = arr1+arr2
diff = arr1-arr2
mul = arr1*arr2
div = arr1/arr2

print(sum)
print(diff)
print(mul)
print(div)
```

```
[5 7 9]
[-3 -3 -3]
[ 4 10 18]
[0.25 0.4  0.5 ]
```

13.Create a 10×10 NumPy array of random values and subtract the row-wise mean using broadcasting.

```python
import numpy as np

# Create 10×10 random array
arr = np.random.rand(10,10)
# Calculate row-wise mean
row_mean = np.mean(arr, axis=1)

# Subtract row-wise mean using broadcasting
result = arr - row_mean.reshape(10,1)

print(result)
```

```
[[ 0.09931539  0.3007681   0.34277142 -0.21384321 -0.27682533 -0.16017275
   0.31031311 -0.50020904 -0.35076421  0.44864652]
 [-0.2403141  -0.12704143  0.04999194 -0.09066646  0.31969509 -0.16484336
  -0.05500332 -0.03008382  0.58306973 -0.24480427]
 [ 0.34081766 -0.21685163 -0.06832773 -0.26488332  0.03570529  0.1382415
  -0.43051903  0.22529198  0.17804966  0.06247561]
 [ 0.23102974 -0.18997534 -0.11110499  0.27022371  0.28789143 -0.20733574
  -0.45953934 -0.4176884   0.52540098  0.07109795]
 [-0.17867548 -0.07236531  0.21017688  0.34357391 -0.45495095  0.12020012
   0.29943939 -0.30994023  0.27462095 -0.23207928]
 [ 0.04318113 -0.15647296 -0.18292823 -0.42093184  0.37203712 -0.137892
   0.1937703  -0.14215184 -0.03443609  0.46582441]
 [ 0.16596587 -0.25145728  0.01675177  0.08155505 -0.34985083  0.36585593
   0.22845142 -0.34968393  0.26053461 -0.16812261]
 [ 0.01494364 -0.24649222  0.15224369  0.42864273 -0.23337913  0.25133146
  -0.40396834  0.04699033 -0.06773353  0.05742138]
 [-0.3998342  -0.36837729 -0.14790238 -0.05007901  0.5810941  -0.27495877
   0.00515039  0.45617805 -0.01865097  0.21738009]
 [-0.21035719 -0.0686933  -0.47525368  0.44148839 -0.49550828  0.42928108
   0.41702899 -0.44776127  0.33158     0.07819526]]
```

14.Implement element-wise multiplication of two matrices using broadcasting.

```python
# task_14

A = np.array([[1,2,3],[4,5,6]])
B = np.array([2,2,2])

result = A*B

print(result)
```

```
[[ 2  4  6]
 [ 8 10 12]]
```

15. Implement Mean Squared Error (MSE) using NumPy:
y_true = np.array([3.0, -0.5, 2.0, 7.0]) y_pred = np.array([2.5, 0.0, 2.0, 8.0])

```python
# task_15

y_true = np.array([3.0,-0.5,2.0,7.0])
y_pred = np.array([2.5,0.0,2.0,8.0])

mse = np.mean((y_true-y_pred)**2)

print("MSE =",mse)
```

```
MSE = 0.375
```

16. Identify and replace missing values (NaN) in the following NumPy array with the mean: data = np.array([1, 2, np.nan, 4, 5, np.nan, 7])

```
# task_16

data = np.array([1,2,np.nan,4,5,np.nan,7])

mean = np.nanmean(data)

data[np.isnan(data)] = mean

print(data)
```

```
[1.  2.  3.8 4.  5.  3.8 7. ]
```

17.Implement Min-Max Scaling on the following array: arr = np.array([10, 20, 30, 40, 50]) Normalize values between 0 and 1.

```
# task_17

arr = np.array([10,20,30,40,50])

scaled = (arr-np.min(arr))/(np.max(arr)-np.min(arr))

print(scaled)
```

```
[0.   0.25 0.5  0.75 1.  ]
```

International Islamic University Islamabad
Faculty of Engineering and Technology
Department of Electrical Engineering

Artificial Intelligence & Machine Learning LAB

Lab No. 2

Implementing Numerical Data Operations Using NumPy

Name: Bushra Nazir **Reg. No.:** 1071-F22

S. No.	Criteria	Allocated Percentage	Level 1 (0%)	Level 2 (25%)	Level 3 (50%)	Level 4 (75%)	Level 5 (100%)	Marks Obtained
(A) PSYCHOMOTOR (To be judged in the lab during experiment)								
1	Practical Implementation OR Connectivity/Arangement of Equipment	5	Absent	***Degree of errors in applying procedural knowledge or Degree of errors in arrangement of equipment:***				
			0	With several critical errors, Imcomplete and Not Neat	With few errors, Incomplete and not Neat	With some errors, Complete but not Neat	Without errors, Complete and Neat	
				1.25	2.5	3.75	5	
2	Use of Equipment OR Use of Simulation/Programming Tool	2	Absent	***Use of tools, equipment and materials with:***				
			0	Limited Competence	Some Competence	Considerable Competence	Good Competence	
				0.5	1	1.5	2	
3	Safety (Optional)	0	Absent	***Degree of reminders to follow safety procedure with:***				
			0	Rarely Follow Safety Rules	Needs Reminders	Needs minor Reminders	Follows Safety Rules	
				0	0	0	0	
	SUB-TOTAL (PT)	**7**	**SUB-TOTAL MARKS OBTAINED (PO)**					
(B) COGNITIVE (To be judged on the copy of experiment submitted)								
4	Data Record, Analysis and Evaluation OR Algorithm Design	1	Absent	***Data record, Analysis and Evaluation or Designed Algorithm is:***				
			0	Incorrect /Incomplete	Complete with some errors	Complete with few errors	Complete and Accurate	
				0.25	0.5	0.75	1	
5	Procedural Awareness (Optional)	0	Absent	***Procedural awareness of the completed task is:***				
			0	Inappropriate	Appropriate with some errors	Appropriate with few errors	Appropriate	
				0	0	0	0	
	SUB-TOTAL (CT)	**1**	**SUB-TOTAL MARKS OBTAINED (CO)**					
(C) AFFECTIVE (To be judged in the lab during experiment)								
6	Level of Participation & Attitude to Achieve Individual/Group Goals	2	Absent	***Degree of positive interaction as an individual or within a group:***				
			0	Rare interaction	Some interaction	Acceptable interaction	Good interaction	
				0.5	1	1.5	2	
	SUB-TOTAL (AT)	**2**	**SUB-TOTAL MARKS OBTAINED (AO)**					

Total Marks (PT + CT + AT) : 10

Marks Obtained (PO + CO + AO) :

Instructor